

Sprache: Python 3 (pandas, NumPy, seaborn, matplotlib, yfinance, SciPy) · Repository:
github.com/phile2006/Stock-Correlation-Heatmap

1 – Ziel des Projekts

Es soll ein Kommandozeilen-Tool entwickelt werden, das für eine frei wählbare Liste von Aktien und ETFs die **Korrelation der Renditen** berechnet und als farbcodierte **Heatmap** (PNG) visualisiert. Stark korrelierte Paare bewegen sich verhältnismäßig gleich; niedrig oder negativ korrelierte Werte sind Kandidaten für die **Diversifikation** eines Portfolios — genau diese Paare soll das Tool zusätzlich in der Konsole hervorheben.

Das Projekt trainiert den Umgang mit echten, unvollkommenen Finanzdaten (Handelskalender, fehlende Tage, verschiedene Börsen), den Python-Data-Stack (pandas, NumPy, seaborn) und die korrekte statistische Aufbereitung: Korrelation auf Renditen statt Kursen, Wahl der Korrelationsmethode, Clustering der Matrix.

2 – Fachlicher Hintergrund

Warum Renditen statt Kurse?

Zwei Aktien, die beide langfristig steigen, hätten auf Kursbasis fast immer eine Korrelation nahe 1 — der gemeinsame Trend dominiert alles. Aussagekräftig ist die Korrelation der **täglichen Renditen**: Sie misst, ob sich zwei Werte *am selben Tag* in dieselbe Richtung bewegen. Zwei Varianten:

$$\text{einfache Rendite: } r_t = P_t/P_{t-1} - 1 \quad \cdot \quad \text{Log-Rendite: } r_t = \ln(P_t/P_{t-1})$$

Bei Tagesdaten sind beide praktisch identisch; Log-Renditen sind über die Zeit additiv und daher der Standard.

Korrelationsmethoden

- **Pearson**: misst den linearen Zusammenhang; Standard, aber empfindlich gegen Ausreißer.
- **Spearman**: Rang-Korrelation; robuster gegen Ausreißer und nichtlineare, aber monotone Zusammenhänge. Sinnvoller Vergleichswert bei volatilen Einzelaktien.

Interpretation & Fallstricke

- Korrelationen sind **nicht stabil**: In Stressphasen steigen sie typischerweise stark an („correlation goes to 1“) — Diversifikation hilft genau dann am wenigsten.
- Unter ~60 gemeinsamen Beobachtungen sind die Werte statistisch wenig belastbar — das Tool soll warnen.
- Ticker verschiedener Börsen (US + Xetra) überlappen nur an gemeinsamen Handelstagen; unterschiedliche Handelszeiten können Tages-Korrelationen systematisch *unterschätzen*.

3 – Aufgabenstellung

Teilaufgabe 1 — Datenbeschaffung

- Lade bereinigte Schlusskurse (adjusted close) über `yfinance`; Standarduniversum: US-Mega-Caps (AAPL, MSFT, NVDA, AMZN, GOOGL, META, TSLA, JPM, XOM) plus ETFs als Vergleichsanker (SPY, QQQ) und Diversifizierer (GLD, TLT).
- Unterstütze wahlweise `--period` (z.B. 1y, 2y) oder festen Bereich `--start/--end`; Intervalle 1d, 1wk, 1mo.
- Richte alle Serien auf gemeinsame Handelstage aus (Zeilen mit Lücken verwerfen) und brich mit klarer Fehlermeldung ab, wenn ein Ticker keine Daten liefert.

Teilaufgabe 2 — Berechnung

- Berechne Renditen (log Standard, simple optional) und daraus die Korrelationsmatrix (pearson Standard, spearman optional).
- Gib die Anzahl der gemeinsamen Beobachtungen aus; warne unter 60.

Teilaufgabe 3 — Visualisierung

- Zeichne die Matrix als Heatmap (seaborn): **rot** = positiv, **blau** = negativ, **weiß** = unkorreliert; Werte in den Zellen annotiert (abschaltbar mit `--no-annot`).
- **Cluster-Sortierung** (Standard): Ordne die Ticker per hierarchischem Clustering (SciPy-Linkage auf der Distanz 1 – Korrelation), sodass korrelierte Gruppen als Blöcke entlang der Diagonale sichtbar werden. Alternativ `alpha` oder `none`.
- Speichere als PNG (`--output, Standard heatmap.png`).

Teilaufgabe 4 — Konsolen-Report

- Gib die fünf am **niedrigsten** korrelierten Paare aus (Diversifikationskandidaten) und die fünf am **höchsten** korrelierten Paare (Klumpenrisiko).

Teilaufgabe 5 — CLI

Option	Standard	Bedeutung
<code>--tickers</code>	Mega-Caps + ETFs	Liste der Ticker (yfinance-Symbole, Xetra mit .DE)
<code>--period</code>	1y	Zeitraum (6mo, 1y, 2y, 5y, max, ...)
<code>--start / --end</code>	—	fester Datumsbereich YYYY-MM-DD (überschreibt <code>--period</code>)
<code>--interval</code>	1d	Datenintervall: 1d, 1wk, 1mo
<code>--returns</code>	log	Renditeberechnung: log oder simple
<code>--method</code>	pearson	Korrelationsmethode: pearson oder spearman

Option	Standard	Bedeutung
--sort	cluster	Sortierung: cluster, alpha, none
--output	heatmap.png	Name der Ausgabedatei
--no-annot	—	Zellenwerte ausblenden (bei vielen Tickern übersichtlicher)

4 – Technische Anforderungen

- Ein einziges, klar gegliedertes Skript `correlation_heatmap.py` plus `requirements.txt`; Installation in einem venv.
- Farbskala symmetrisch um 0 fixieren (−1 bis +1), damit Farben zwischen Läufen vergleichbar sind.
- Fehlerfälle (unbekannter Ticker, leerer Datenbereich, nur 1 Ticker) mit verständlicher Meldung abbrechen.

5 – Abnahmekriterien & Verifikation

Das Projekt gilt als fertig, wenn:

- **Plausibilität:** SPY und QQQ sind hoch korreliert (>0.9); GLD und TLT haben deutlich niedrigere Korrelation zu den Aktien — sie erscheinen als eigene Blöcke in der Cluster-Sortierung.
- **Symmetrie:** Die Matrix ist symmetrisch mit Diagonale 1.0.
- **Methodenvergleich:** Pearson- und Spearman-Ergebnisse sind ähnlich, aber nicht identisch.
- Ein Lauf mit deutschen Tickern (z. B. SAP.DE SIE.DE) funktioniert und reduziert sichtbar die gemeinsame Datenbasis (Konsole zeigt weniger Beobachtungen).
- PNG und Konsolen-Report entstehen in einem einzigen Aufruf ohne manuelle Nacharbeit.

6 – Mögliche Erweiterungen

- Rollierende Korrelation (z. B. 60-Tage-Fenster) als Zeitreihen-Plot für ein ausgewähltes Paar — macht die Instabilität aus 2.3 sichtbar.
- Export der Matrix als CSV für Weiterverarbeitung (z. B. im Portfolio-Optimizer, Projekt 3).
- Dendrogramm des Clusterings neben der Heatmap darstellen.